

ФГАОУ ВО «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и
управления»

ОТЧЁТ

по дисциплине «СУБД»

на тему: «Работа с СУБД PostgreSQL»

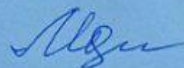
студента Нурсалимова Данияра Ерлановича группы ПИ-221

Пояснительная записка

Шифр работы ЛР-02068999-43-ПИ-221-01 ПЗ

Направление 09.03.03 Прикладная информатика

Старший преподаватель



М.П. Маркова

Студент



Д. Е. Нурсалимов

Омск 2024

Введение

Теоретические сведения

18. Служба, работающая с замечаниями машиниста электропоезда

Основные задачи, которое решает служба. Сбор замечаний машиниста, доведение их до конкретных служб, контроль за исправлением, анализ.

Описание предметной области. При движении электропоезда все свои замечания машинист заносит в журнал. По окончании работы данные из журнала формализуются и заносятся в компьютер. Реализация по замечаниям закрепляется за конкретной службой (пути, электрообеспечения и пр.) и этой службе дается срок на составление плана мероприятий. Этот план контролируется, и после полного завершения работ делается отметка об исправлении замечаний. Должна поддерживаться информация об участках железной дороги (например, Барабинский, Новосибирский и т.д.), машинистах и их замечаниях, соответствующих службах железной дороги и мероприятиях по замечаниям.

Лабораторная работа № 1

Проектирование базы данных с использованием ER-технологии

Практическая работа

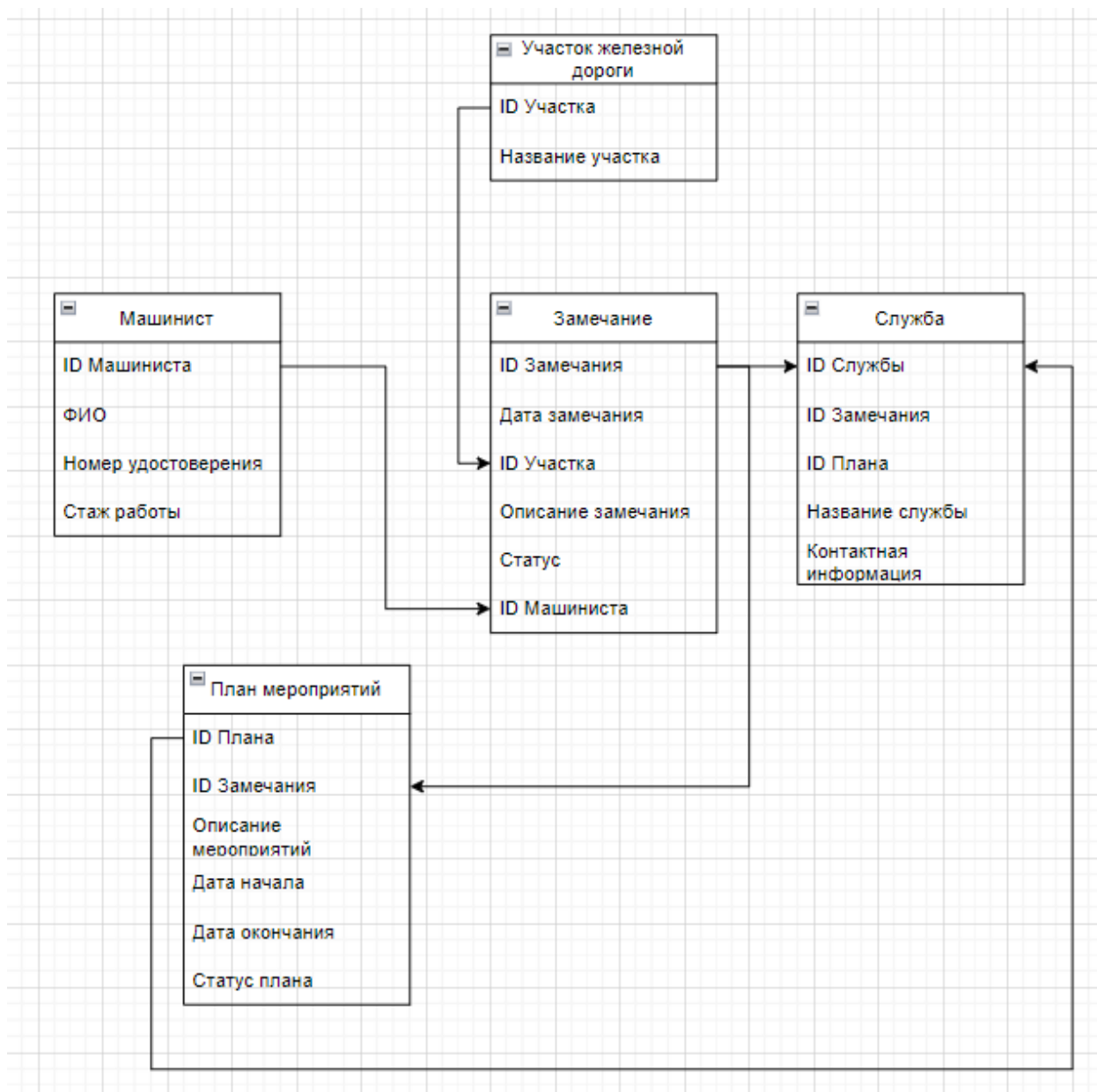


Рис.1 – Логическая модель данных предметной области

Лабораторная работа № 2

Создание новой базы данных в среде PostgreSQL

Практическая работа

Создание базы данных под названием insurance (страховки).

```
SQL Shell (psql)
postgres=# psql
postgres=# psql -d
postgres=# CREATE DATABASE insurance;
ОШИБКА: ошибка синтаксиса (примерное положение: "psql")
СТРОКА 2: psql
      ^
postgres=# CREATE DATABASE insurance;
CREATE DATABASE
postgres=# \l
```

database	encoding	lc_collate	lc_ctype	provider
insurance	postgres	UTF8	Russian_Russia.1251	libc
postgres	postgres	UTF8	Russian_Russia.1251	libc
template0	postgres	UTF8	Russian_Russia.1251	libc
template1	postgres	UTF8	Russian_Russia.1251	libc

Рис.1 – Командная строка создание базы данных

Подключение к базе данных через консоль.

```
postgres=# \c insurance
неверная команда \c
Введите \? для получения справки.
postgres=# \c insurance
Вы подключены к базе данных "insurance" как пользователь "postgres".
insurance=# \conninfo
Вы подключены к базе данных "insurance" как пользователь "postgres" (сервер "localhost": адрес "::1", порт "5432").
insurance=#
```

Рис.2 – Командная строка подключение к базе данных

Проверка создания базы через графическое приложение pgAdmin4.

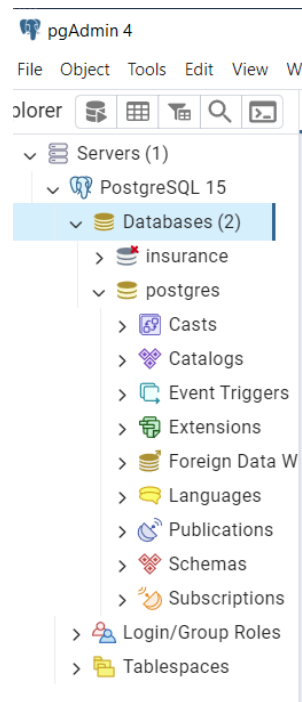


Рис.3 – Список баз в приложении PostgreSQL

На рисунке 4 показаны графики работы базы.

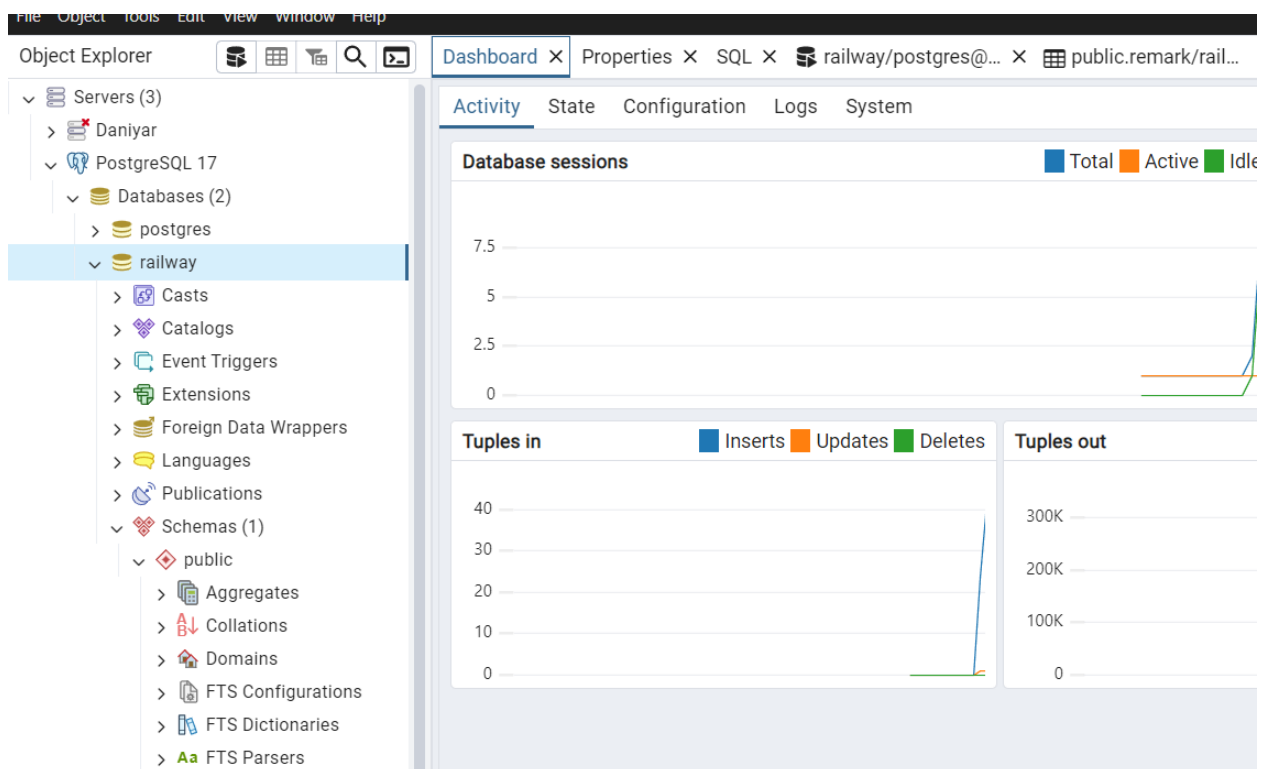
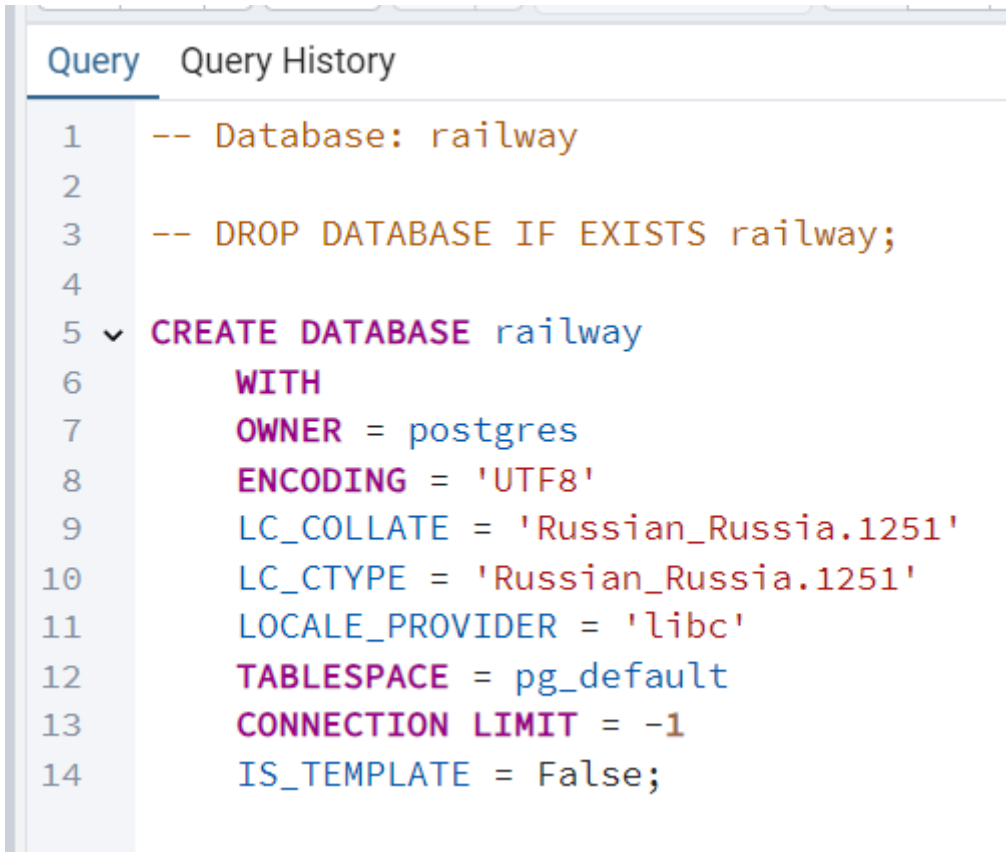


Рис.4 – Демонстрация работы созданной базы

Настройки базы «Железная дорога».



```
Query  Query History
1  -- Database: railway
2
3  -- DROP DATABASE IF EXISTS railway;
4
5  ✓ CREATE DATABASE railway
6      WITH
7      OWNER = postgres
8      ENCODING = 'UTF8'
9      LC_COLLATE = 'Russian_Russia.1251'
10     LC_CTYPE = 'Russian_Russia.1251'
11     LOCALE_PROVIDER = 'libc'
12     TABLESPACE = pg_default
13     CONNECTION LIMIT = -1
14     IS_TEMPLATE = False;
```

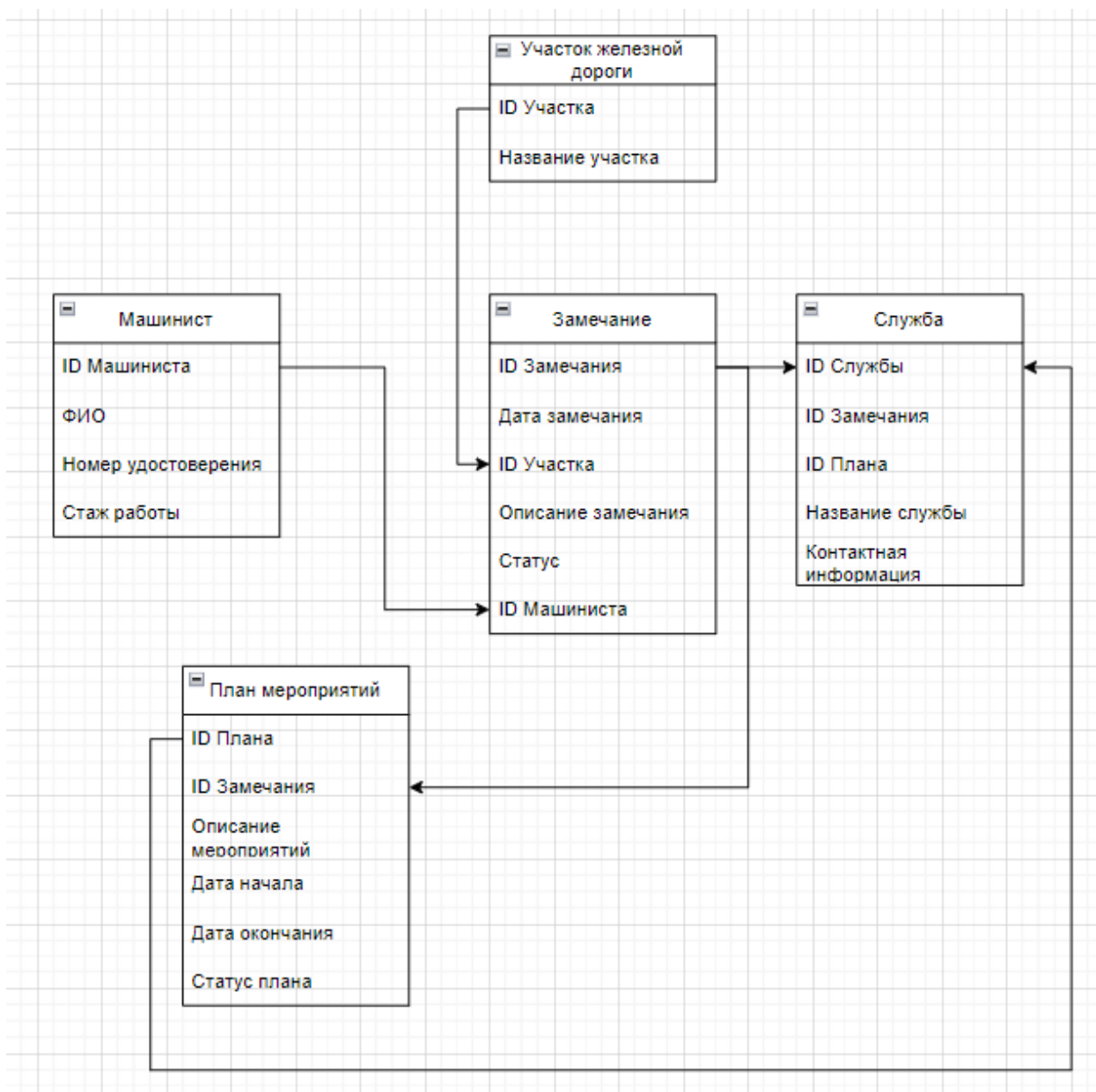
Рис.5 – Настройки созданной базы

Лабораторная работа №3

Практическая работа

При выполнении задания необходимо для заданной предметной области:

- ☐ создать таблицы, определить поля таблиц и тип полей;
- ☐ определить связи между таблицами и ограничения целостности;
- ☐ составить отчет.



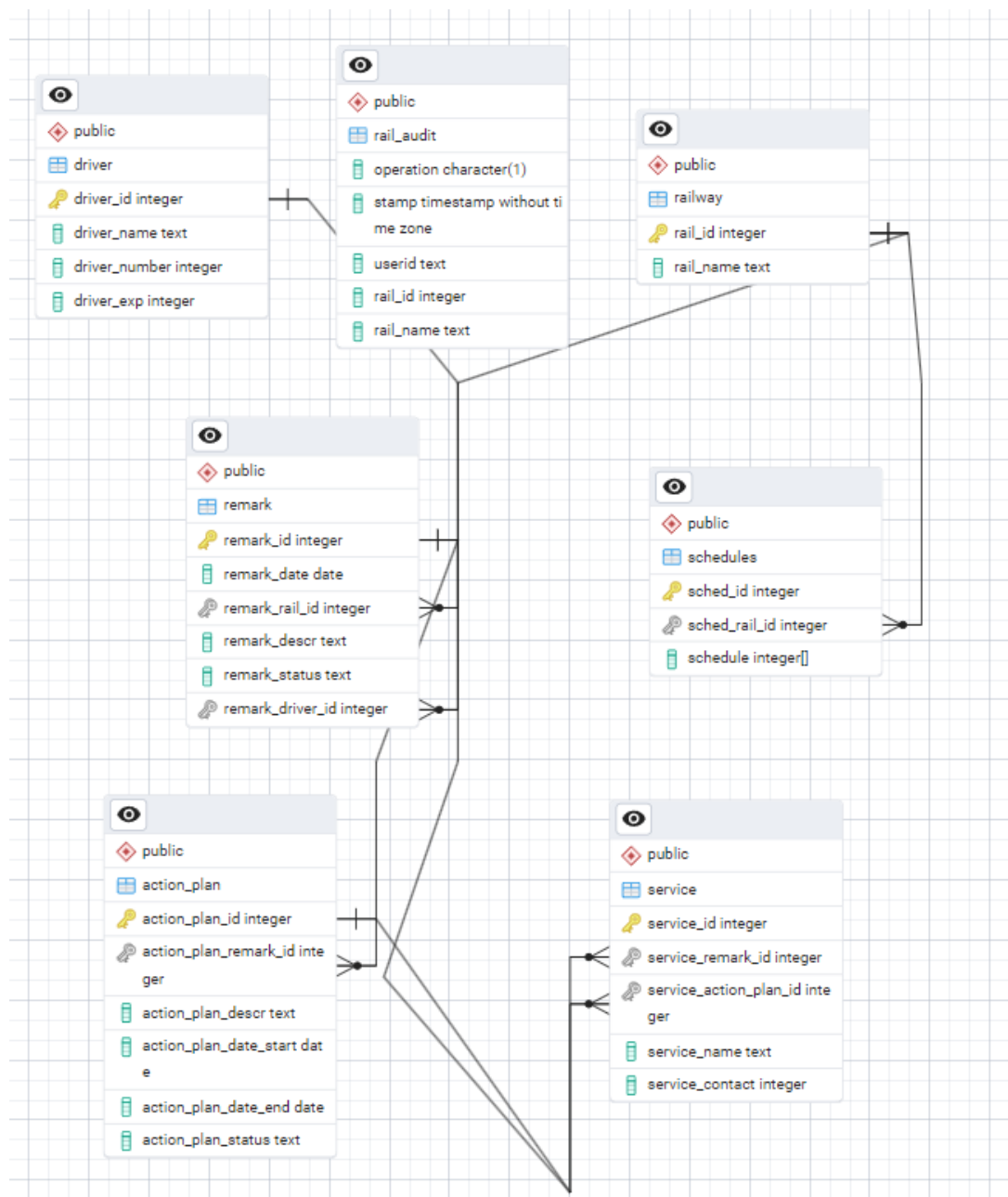
C: > Users > Данияр > Desktop >  railway.sql

```
1 CREATE TABLE railway(  
2     rail_ID integer NOT NULL,  
3     rail_name text NOT NULL,  
4     CONSTRAINT railway_PK PRIMARY KEY (rail_ID)  
5 );  
6  
7 CREATE TABLE remark(  
8     remark_ID integer NOT NULL,  
9     remark_date date NOT NULL,  
10    remark_rail_ID integer,  
11    remark_descr text NOT NULL,  
12    remark_status text NOT NULL,  
13    remark_driver_ID integer NOT NULL,  
14    CONSTRAINT remark_PK PRIMARY KEY (remark_ID)  
15 );  
16  
17 CREATE TABLE driver(  
18     driver_ID integer NOT NULL,  
19     driver_name text NOT NULL,  
20     driver_number integer NOT NULL,  
21     driver_exp integer NOT NULL,  
22     CONSTRAINT driver_PK PRIMARY KEY (driver_ID)  
23 );  
24  
25 CREATE TABLE service(  
26     service_ID integer NOT NULL,  
27     service_remark_ID integer NOT NULL,  
28     service_action_plan_ID integer NOT NULL,  
29     service_name text NOT NULL,  
30     service_contact integer NOT NULL,  
31     CONSTRAINT service_PK PRIMARY KEY (service_ID)  
32 );
```

```

33
34 CREATE TABLE action_plan(
35     action_plan_ID integer NOT NULL,
36     action_plan_remark_ID integer NOT NULL,
37     action_plan_descr text NOT NULL,
38     action_plan_date_start date NOT NULL,
39     action_plan_date_end date NOT NULL,
40     action_plan_status text NOT NULL,
41     CONSTRAINT action_plan_PK PRIMARY KEY (action_plan_ID)
42 )
43

```



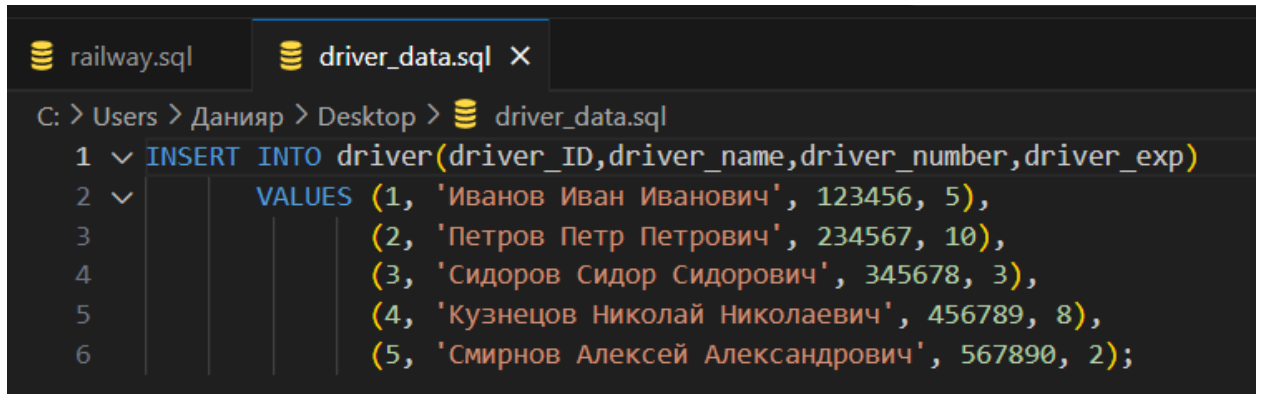
Лабораторная работа № 4

ВСТАВКА, УДАЛЕНИЕ И ОБНОВЛЕНИЕ ДАННЫХ

Практическая работа

При выполнении задания, используя инструкции SQL:

- заполнить согласованными данными таблицы базы данных;
- при необходимости исправить введенную информацию;
- составить отчет.



```
railway.sql driver_data.sql X
C: > Users > Данияр > Desktop > driver_data.sql
1  INSERT INTO driver(driver_ID,driver_name,driver_number,driver_exp)
2  VALUES (1, 'Иванов Иван Иванович', 123456, 5),
3         (2, 'Петров Петр Петрович', 234567, 10),
4         (3, 'Сидоров Сидор Сидорович', 345678, 3),
5         (4, 'Кузнецов Николай Николаевич', 456789, 8),
6         (5, 'Смирнов Алексей Александрович', 567890, 2);
```

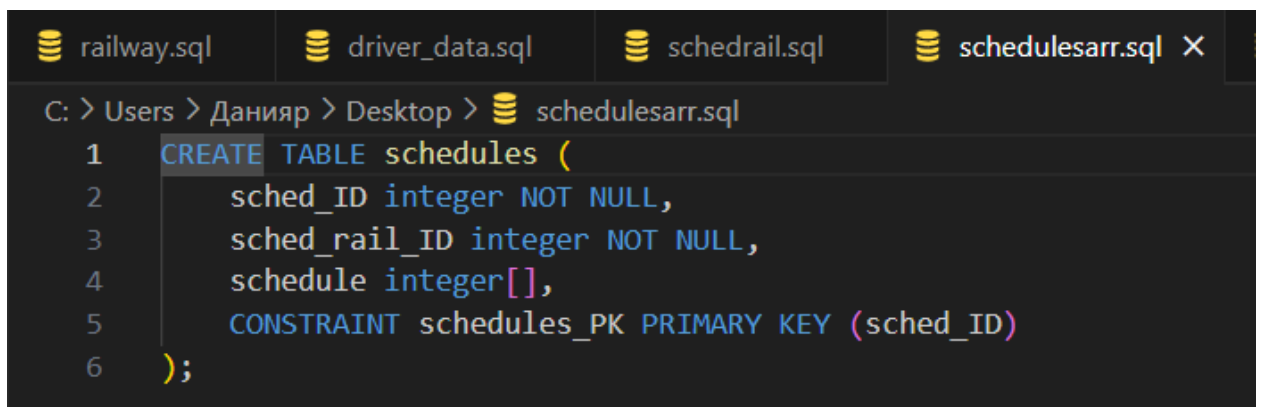
Лабораторная работа No 5

РАБОТА С МАССИВАМИ И ТИПАМИ JSON

Практическая работа:

Необходимо для созданной базы данных:

- добавить таблицу со столбцом, содержащим массив значений;
- добавить таблицу со столбцом, содержащим значения в формате JSON;



```
railway.sql driver_data.sql schedrail.sql schedulesarr.sql X
C: > Users > Данияр > Desktop > schedulesarr.sql
1  CREATE TABLE schedules (
2      sched_ID integer NOT NULL,
3      sched_rail_ID integer NOT NULL,
4      schedule integer[],
5      CONSTRAINT schedules_PK PRIMARY KEY (sched_ID)
6  );
```

```
C: > Users > Данияр > Desktop > alterschedulesarr.sql
1 ALTER TABLE schedules
2 ADD CONSTRAINT schedules_FK FOREIGN KEY (sched_rail_ID)
3 REFERENCES railway (rail_ID)
4 ON DELETE CASCADE
5 ON UPDATE CASCADE;
```

```
C: > Users > Данияр > Desktop > dataschedulesarr.sql
1 INSERT INTO schedules VALUES
2 (1,1, '{1,3,5,6,7}'::integer[]),
3 (2,2, '{1,2,5,7}'::integer[]),
4 (3,3, '{2,5}'::integer[]),
5 (4,4, '{3,5,6}'::integer[]),
6 (5,5, '{2,4}'::integer[]);
```

```
1 SELECT schedules.sched_ID, railway.rail_name, schedules.schedule
2 FROM schedules JOIN railway
3 ON schedules.sched_rail_ID = railway.rail_ID;
```

Data Output Messages Notifications

	sched_id integer	rail_name text	schedule integer[]
1	1	Участок 1: Москва - Санкт-Петербург	{1,3,5,6,7}
2	2	Участок 2: Новороссийск - Краснод...	{1,2,5,7}
3	3	Участок 3: Казань - Уфа	{2,5}
4	4	Участок 4: Владивосток - Хабаровск	{3,5,6}
5	5	Участок 5: Екатеринбург - Челябинск	{2,4}

Лабораторная работа No 6

СОЗДАНИЕ ЗАПРОСОВ НА ВЫБОРКУ

Практическая работа:

Необходимо для созданной базы данных:

- построить запрос на выборку с использованием какого-либо соединения;
- построить запрос на выборку, содержащий вложенный запрос;

Query Query History

```
1 SELECT railway.rail_ID, COUNT(remark.remark_ID) AS total
2 FROM railway LEFT JOIN remark
3 ON railway.rail_ID = remark.remark_rail_ID
4 GROUP BY railway.rail_ID
5 HAVING COUNT(remark.remark_ID) >= 2
6 ORDER BY total DESC, railway.rail_ID ASC;
```

Data Output Messages Notifications

rail_id	total
[PK] integer	bigint

railway/postgres@PostgreSQL 17

Query Query History

```
1 SELECT railway.rail_ID, COUNT(remark.remark_ID) AS total
2 FROM railway LEFT JOIN remark
3 ON railway.rail_ID = remark.remark_rail_ID
4 GROUP BY railway.rail_ID
5 HAVING COUNT(remark.remark_ID) >= 2
6 ORDER BY total DESC, railway.rail_ID ASC;
```

Data Output Messages Notifications

	rail_id	total
	[PK] integer	bigint
1	1	1
2	2	1
3	3	1
4	4	1
5	5	1

Лабораторная работа No 7

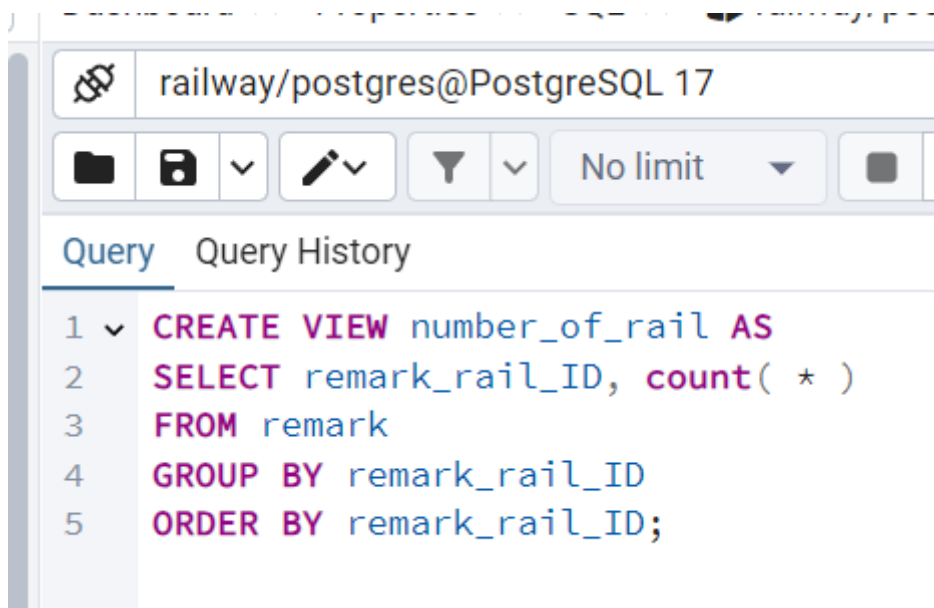
ПРЕДСТАВЛЕНИЯ

Практическая работа:

Необходимо для созданной базы данных:

- создать несколько простых и материализованных представлений и включить их в базу данных;
- сформировать запросы к этим представлениям;
- составить отчет.

Простое представление для агентов с зарплатой выше среднего:



Лабораторная работа No 8

ФУНКЦИИ И ПРОЦЕДУРЫ

Практическая работа:

Необходимо для созданной базы данных:

- написать несколько хранимых процедур и включить их в базу данных;
- составить отчет.

Создание и вызов процедуры «Добавление участка ж\д»

```
Query Query History
1 CREATE OR REPLACE PROCEDURE rail_insert
2   (rails_ID integer, rails_name text)
3   LANGUAGE sql
4   AS $$
5     INSERT INTO railway (
6       rail_ID, rail_name)
7     VALUES (
8       rails_ID, rails_name);
9   $$;
```

```
Query Query History
1 CALL rail_insert(7, 'Пограничная');
```

	rail_id [PK] integer	rail_name text
1	1	Участок 1: Москва - Санкт-Петербург
2	2	Участок 2: Новороссийск - Краснодар...
3	3	Участок 3: Казань - Уфа
4	4	Участок 4: Владивосток - Хабаровск
5	5	Участок 5: Екатеринбург - Челябинск
6	7	Пограничная

Лабораторная работа No 9

ТРАНЗАКЦИИ

Практическая работа:

Необходимо для созданной базы данных:

- создать транзакцию с точкой сохранения в режиме явных транзакций, произвести откат к точке сохранения и фиксацию;
- составить отчет.

Транзакция позволяет сохранить весь результат строк до ошибки и ставить сохранения.

Query

Query History

1

BEGIN;

2

▼

INSERT INTO railway (rail_ID, rail_name)

3

VALUES (217, 'Морской');

4

▼

INSERT INTO railway (rail_ID, rail_name)

5

VALUES (218, 'Добровольственный');

6

SAVEPOINT point_one;

7

▼

INSERT INTO railway (rail_ID, rail_name)

8

VALUES (219, '145-ый километр');

9

▼

INSERT INTO railway (rail_ID, rail_name)

10

VALUES (220, 'Коронный');

11

ROLLBACK TO SAVEPOINT point_one;

12

COMMIT;

Data Output

Messages

Notifications

COMMIT

Query

Query History

1

SELECT * FROM railway

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	rail_id [PK] integer	rail_name text
1	1	Участок 1: Москва - Санкт-Петербург
2	2	Участок 2: Новороссийск - Краснодар...
3	3	Участок 3: Казань - Уфа
4	4	Участок 4: Владивосток - Хабаровск
5	5	Участок 5: Екатеринбург - Челябинск
6	7	Пограничная
7	217	Морской
8	218	Добровольственный

Лабораторная работа No 10

КУРСОРЫ

Практическая работа:

Необходимо для созданной базы данных:

- решить несколько задач с использованием курсоров, включив курсор в хранимую процедуру или функцию;
- составить отчет.

Курсоры позволяют выбрать в данные из нескольких таблиц, и записывать их в хранимую процедуру или функцию

```
Query Query History
1  CREATE OR REPLACE PROCEDURE remark_select (
2      IN remark_id integer,
3      IN rail_id integer,
4      IN driver_id integer)
5      LANGUAGE 'plpgsql'
6      AS $$
7      DECLARE
8          c_remark_ID integer; c_rail_ID integer; c_rail_name text;
9          c_driver_ID integer; c_driver_name text; c_remark_date date;
10     cur CURSOR FOR SELECT
11         r.remark_ID, r.remark_rail_ID, c.rail_name,
12         r.remark_driver_ID, d.driver_name, r.remark_date
13     FROM railway c JOIN remark r ON c.rail_ID = r.remark_rail_ID
14         JOIN driver d ON d.driver_ID = r.remark_driver_ID
15     WHERE (remark_ID IS NULL OR r.remark_ID = remark_ID) AND
16         (rail_ID IS NULL OR r.remark_rail_ID = rail_ID) AND
17         (driver_ID IS NULL OR r.remark_driver_ID = driver_ID);
18     BEGIN
19         OPEN cur;
20     LOOP
21         FETCH cur INTO c_remark_ID, c_rail_ID, c_rail_name, c_driver_ID,
22             c_driver_name, c_remark_date;
23         EXIT WHEN not found;
24     RAISE NOTICE 'Курсор: %, %, %, %, %, %', c_remark_ID, c_rail_ID, c_rail_name, c_driver_ID,
25         c_driver_name, c_remark_date;
26     END LOOP;
27     CLOSE cur;
28     END;
29     $$;

31     RAISE NOTICE 'Контракт: %, №Клиента: %, Имя: %, №Агента: %, Имя: %, Дата: %', c_cont_id,
32         c_clt_name, c_ag_id, c_ag_name, c_cont_data;
33     END LOOP;
34     CLOSE cur;
35     END;
36     $$;
```

Лабораторная работа No 11

Триггеры

Практическая работа:

Необходимо для созданной базы данных:

- написать два триггера (можно ограничиться триггерами, срабатывающими при модификации данных);
- составить отчет.

Триггер срабатывает при модификации данных таблицы client и сохраняет операции в таблице client_audit

The session is idle and there is no current transaction.

```

1 CREATE TABLE rail_audit(
2     operation char(1) NOT NULL,
3     stamp timestamp NOT NULL,
4     userid text NOT NULL,
5     rail_ID integer NOT NULL,
6     rail_name text NOT NULL
7 );

```

```

1 CREATE OR REPLACE FUNCTION process_rail_audit()
2 RETURNS TRIGGER
3 AS $$
4 BEGIN
5     IF (TG_OP = 'DELETE') THEN
6         INSERT INTO rail_audit SELECT 'D', now(), user, OLD.*;
7     ELSEIF (TG_OP = 'UPDATE') THEN
8         INSERT INTO rail_audit SELECT 'U', now(), user, NEW.*;
9     ELSEIF (TG_OP = 'INSERT') THEN
10        INSERT INTO rail_audit SELECT 'I', now(), user, NEW.*;
11    END IF;
12    RETURN NULL;
13 END;
14 $$ LANGUAGE plpgsql;

```

```

1 CREATE TRIGGER rail_audit
2 AFTER INSERT OR UPDATE OR DELETE ON railway
3 FOR EACH ROW EXECUTE FUNCTION process_rail_audit();

```

```

1 INSERT INTO railway VALUES (78, 'Отрадный');
2 SELECT * FROM railway
3 ORDER BY rail_ID ASC;

```

Query Query History

```

1 INSERT INTO railway VALUES (78, 'Отрадный');
2 SELECT * FROM railway
3 ORDER BY rail_ID ASC;

```

Data Output Messages Notifications

	rail_id [PK] integer	rail_name text
1	1	Участок 1: Москва - Санкт-Петербург
2	2	Участок 2: Новороссийск - Краснод...
3	3	Участок 3: Казань - Уфа
4	4	Участок 4: Владивосток - Хабаровск
5	5	Участок 5: Екатеринбург - Челябинск
6	7	Пограничная
7	78	Отрадный
8	217	Морской
9	218	Добровольственный

Query Query History

```

1 SELECT * FROM rail_audit;

```

Data Output Messages Notifications

	operation character (1)	stamp timestamp without time zone	userid text	rail_id integer	rail_name text
1	I	2025-01-21 12:50:59.186282	postgres	78	Отрадный

Заключение

В ходе изучения темы "Создание новой базы данных в среде PostgreSQL" были рассмотрены ключевые теоретические аспекты, касающиеся работы с базами данных. Мы подробно рассмотрели процессы создания, изменения, выбора и удаления баз данных, а также создание, изменение и удаление схем.

Полученные знания позволили мне не только глубже понять архитектуру и функциональность PostgreSQL, но и реализовать практические навыки. В результате успешного выполнения всех этапов создал свою собственную базу данных под названием insurance. Этот проект станет началом возможностей применения теоретических знаний на практике и углубления понимания работы с реляционными базами данных.

Список использованных источников

1. Домбровская, Генриэтта. Оптимизация запросов в PostgreSQL : пер. с англ. Генриэтта Домбровская, Борис Новиков, Анна Бейликова. – М. : ДМК Пресс, 2022. – 278 с.
2. Кригель, Алекс. SQL. Библия пользователя : пер. с англ. / Алекс Кригель, Борис Трухнов. – 2-е изд. – М. : Вильямс, 2010. – 752 с.
3. Лузанов, П.В. Postgres. Первое знакомство / Е.В. Рогов, И.В. Лёвшин. – 6-е изд., перераб. и доп. – М.: Постгрес Профессиональный, 2020. – 178 с.
4. Малков, О. Б. Работа с Transact-SQL [Электронный ресурс] / О. Б. Малков, М. В. Девятерикова. – Омск : Изд-во ОмГТУ, 2015. – 1 эл. опт. диск (DVD-ROM).
5. Малков, О. Б. Oracle SQL. Базовая часть : учеб. пособие [Электронный ресурс] / О. Б. Малков ; Минобрнауки России, ОмГТУ. – Омск : Изд-во ОмГТУ, 2019. – 1 эл. опт. диск (DVD-ROM).
6. Моргунов, Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.
7. Новиков, Б.А. Основы технологий баз данных: учеб. пособие / Б.А. Новиков, Е.А. Горшкова, Н.Г. Графеева; под ред. Е. В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с.
8. Редмонд, Эрик. Семь баз данных за семь недель. Введение в современные базы данных и идеологию NoSQL : пер. с англ. / Эрик Редмонд, Джим. Р. Уилсон; под ред. Жаклин Картер. – М.: ДМК Пресс, 2013. – 384 с.
9. Рогов, Е.В. PostgreSQL 15 изнутри / Е.В. Рогов. – М.: ДМК Пресс, 2023. – 662 с.
10. Dar U., Krosing H., Mlodgenski J., Roybal K. PostgreSQL Server Programming. 2nd Edition Extend. Birmingham, Packt Publishing, 2015. 508 p.
11. PostgreSQL. Documentation. 9th February 2023: PostgreSQL 15.2, 14.7, 13.10, 12.14, and 11.19 Released [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> – Загл. с экрана (дата обращения: 30.03.2023).

12. PostgresPro. Документация к PostgreSQL 15.2 [Электронный ресурс]. – Режим доступа: <https://postgrespro.ru/docs/postgresql/15/> – Загл. с экрана (дата обращения: 30.03.2023).

13. pgAdmin 4. Documentation. Release 7.1. The pgAdmin Development Team May 03, 2023. [Электронный ресурс]. – Режим доступа: <https://www.pgadmin.org/docs/pgadmin4/latest/index.html>. – Загл. с экрана (дата обращения: 10.05.2023).